

Sequential Fine-Tuning Order Fingerprints

Can the final model reveal which task it learned last?

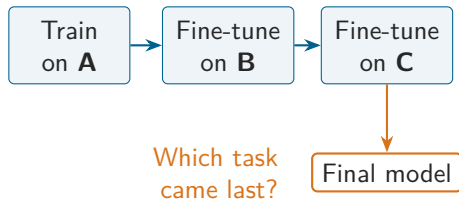
Ahmad Ata Ul Haye, Daniel Flick
PAMI Practice SS 2026, Goethe Universität Frankfurt
30.09.2026

Research question

A network is trained on tasks one after another. Each new task changes the weights.

Does the final checkpoint still carry a measurable trace of the task learned most recently?

Concretely: after training on $A \rightarrow B \rightarrow C$, can we tell that C came last, using only the final model and models trained on a single task?



Setup: three small classification tasks

Each task is a 5-class problem built from CIFAR-100 classes. All tasks share one ResNet18 backbone; every task has its own 5-class head.

Task	Theme	Classes
A	Aquatic animals	beaver, dolphin, otter, seal, whale
B	Electronic / office devices	clock, keyboard, lamp, telephone, television
C	Vehicles	bicycle, bus, motorcycle, pickup truck, train

Six task orders are tested: all permutations of A, B, C.

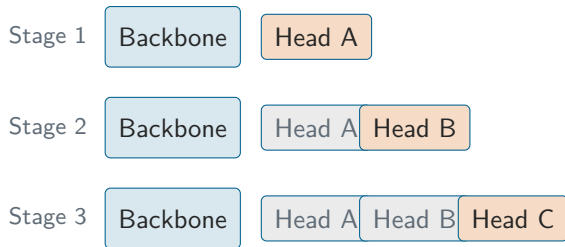
The **actual last task** of an order is simply its final element.

How the sequential models are trained

For an order such as $A \rightarrow B \rightarrow C$:

1. Train backbone and A-head on A.
2. Freeze the A-head, add a B-head, train on B.
3. Freeze A- and B-heads, add a C-head, train on C.

No forgetting protection (no EWC, SI or replay). A checkpoint is saved after every stage; the last one is the **final sequential model**.



Orange = being trained. Grey = frozen head.

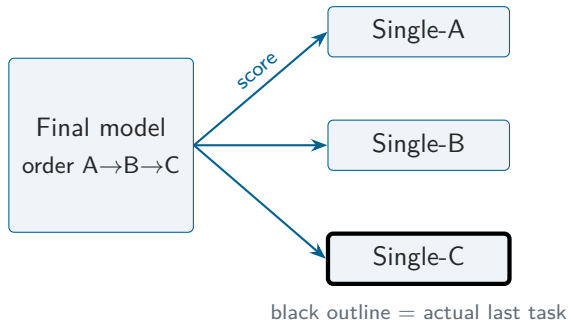
The fingerprint test

Reference models: three models, each trained only on A, B or C (Single-A, Single-B, Single-C).

Test: compare the final sequential model with each reference, and pick the reference that looks most similar. That task is the **predicted last task**.

A prediction is correct if it equals the last element of the order.

Random guessing gives $1/3 = 33.3\%$.



Five families of comparison

Method	What is compared	Prediction rule
Weight distance (L2)	Parameters of final model vs. each reference	Smallest distance
CKA	Feature structure on a fixed image set	Highest similarity
Feature drift	Feature vectors, sample by sample	Smallest drift
Weight matching	Parameters after removing neuron re-ordering; direction of weight change	Smallest matched L2; highest cosine
Fisher trace	Curvature of the final model only (no references)	Rank the three tasks; test both directions

Forgetting, loss barriers and Jacobian sensitivity are computed too, but only as supporting diagnostics.

Weight distance

Euclidean distance between the final model and each reference:

$$D_{\text{full}} = \|\theta_{\text{seq}} - \theta_{\text{ref}}\|_2$$

Two versions:

- **Full model:** backbone and all heads.
- **Backbone only:** the shared ResNet18 parameters, heads excluded.

Rule: the reference with the smallest distance is the predicted last task.

Why two versions?

The full-model distance also includes the heads, which are not comparable in the same way. The backbone version isolates the shared representation.

Representation similarity: CKA and feature drift

Both are computed on the same fixed images. For each image, the 512-dimensional feature vector before the classifier is extracted from the final model and from each reference.

Linear CKA

$$\text{CKA}(X, Y) = \frac{\|X^\top Y\|_F^2}{\|X^\top X\|_F \|Y^\top Y\|_F}$$

X, Y centred feature matrices.

Asks: is the *structure* of the representation similar?

Range 0 to 1. **Highest CKA** predicts the last task.

Feature drift

$$\text{drift}(X, Y) = \text{mean} \left\| \frac{x}{\|x\|} - \frac{y}{\|y\|} \right\|_2$$

Asks: did the individual feature vectors move?

Range 0 to 2. **Lowest drift** predicts the last task.

The fixed evaluation image set

CKA and feature drift are only comparable if every model sees exactly the same images. Two modes exist:

Mode	Images	Details
probe	300	20 random test images for each of the 15 classes; seed 42; created once and reused
all_test	1,500	Every CIFAR-100 test image of the 15 classes; no sampling

The mode is shown in the title of each representation figure. **The results in this presentation use all_test.**

Fisher information: a fingerprint without reference models

The Fisher trace measures how strongly the model's output reacts to weight changes on a task's data. It needs only the **final checkpoint**.

Per task (1,000 samples):

1. Sample a label from the model's own predicted probabilities.
2. Back-propagate the loss; square and accumulate the gradients.
3. Never update the weights.

Result: F_A , F_B , F_C .

Nobody knows in advance whether a *high* value means “recent” or “old”, so both readings are tested:

Hypothesis	Last task
<code>high_is_recent</code>	largest F
<code>low_is_recent</code>	smallest F

Sorting the three values also gives a guess for the full order (chance: 1/6).

Weight matching: a control for neuron reordering

Two networks can compute the same function with neurons in a different order, which inflates L2 distance. Weight matching (Git Re-Basin) reorders the neurons of one model to best fit the other before measuring.

- Solved with the Hungarian algorithm, one permutation group at a time.
- Checked afterwards: the logits of all three heads must be unchanged.
- Also computed: cosine similarity of the *weight updates* $\theta_{\text{final}} - \theta_{\text{init}}$ and $\theta_{\text{task}} - \theta_{\text{init}}$.

All models start from the same initialization, so the expected outcome is the **identity permutation**: matched and unmatched distances are equal. This makes it a **negative control** for the L2 result.

Supporting diagnostics (not predictors)

Forgetting

accuracy right after learning a task minus its final accuracy.

Positive: performance dropped.

Loss barrier

Interpolate linearly between final model and a reference; measure the loss along the path.

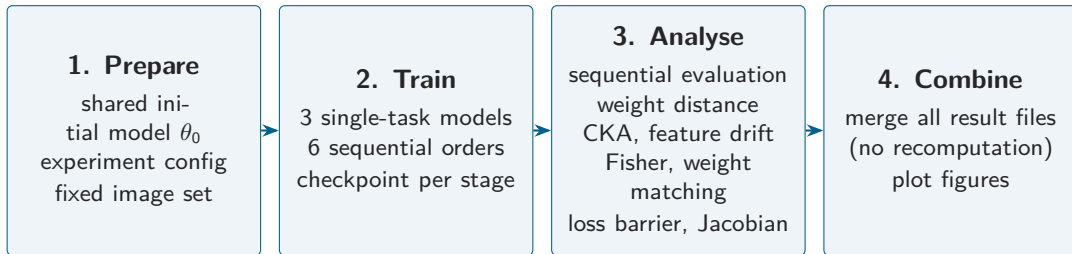
Height = max minus min loss.

Jacobian sensitivity

Gradient norm of the predicted logit with respect to the input image, averaged over images.

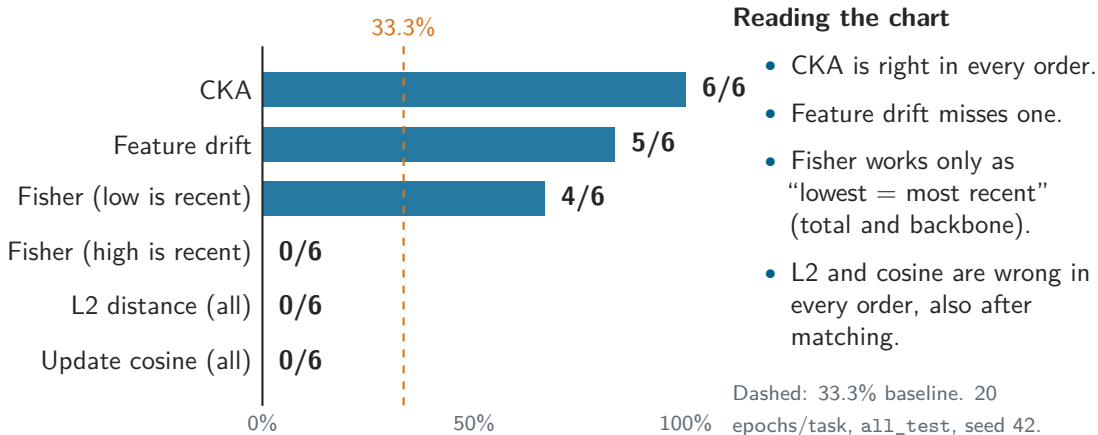
These describe how the models behave. They do not define a rule for the last task, and higher sensitivity does not mean “learned more recently”.

Pipeline



All settings, task orders and class lists come from one config file. Nothing is hard-coded in the scripts.

Result: how often is the last task predicted correctly?



Prediction matrix: every order, every method

Figure not found:
prediction_matrix.png

Rows are orders, columns are methods, cells show the predicted task. **Green** is correct, **red** is wrong.

In every L2 and cosine column the predicted task is the **first** task of the order, never the last.

CKA separates the last task clearly

Figure not found:
cka_similarity.png

One panel per order. Bars show CKA between the final model and Single-A, B, C. Outline marks the actual last task.

- The outlined bar is the highest in all six orders.
- Last-task CKA lies between 0.63 and 0.75.
- The other two references lie between 0.19 and 0.43.

The gap is large, not a near tie.

Feature drift: same direction, smaller margins

Figure not found:
feature_drift.png

Lower drift means more similar features. Outline marks the actual last task.

- The outlined bar is the lowest in 5 of 6 orders.
- The miss is $B \rightarrow C \rightarrow A$: Single-B (0.571) is just below Single-A (0.579).
- All values lie between 0.54 and 0.62, so margins are small compared with CKA.

Weight distance points at the first task

Figure not found:
`weight_distance_scores.png`

L2 distance from each final model to the three references, for the full model (top) and the backbone (bottom). Outline marks the actual last task.

- In every order the **closest** reference is the one of the **first** task.
- The last task is never the closest: 0 of 6.
- Full model and backbone give the same picture.

The project documentation reads this as distance reflecting how far the weights travelled, not recency.

Weight matching changes nothing

Figure not found:
`weight_matching_before_after.png`

Each row is one final model and one reference. Dots: before matching. Crosses: after.

- Dots and crosses coincide everywhere: distances did not change.
- This fits the expected identity permutation with a shared initialization.
- The cosine of weight updates is highest for the first task (about 0.59 to 0.69, others 0.16 to 0.19), so it fails like L2.

Fisher: lowest trace marks the last task in 4 of 6

Figure not found:
fisher_hypothesis_accuracy.png

- low_is_recent: 4/6 for both total and backbone Fisher.
- high_is_recent: 0/6.
- The two misses are the orders that start with A. There, the first task A has the lowest trace and is predicted.
- So this method predicts A in 4 of 6 orders, and A is last in only 2.
- Full training order: 1/6 correct, chance is 1/6.

Forgetting

Figure not found:
forgetting_by_order.png

$\text{forgetting}(T) =$
acc. after learning T –
final acc. on T

- Earlier tasks lose about 0.23 to 0.45 accuracy.
- The last task shows zero forgetting in every order.
- That is by construction: for the last task, “after learning” and “final” are the same checkpoint.

So forgetting gives no independent evidence of a fingerprint.

Hypotheses and outcome

	Hypothesis	Outcome so far
H1	Sequential training leaves a last-task fingerprint	CKA 6/6, feature drift 5/6, Fisher (low is recent) 4/6. L2 and cosine 0/6.
H2	Representation similarity beats weight distance	Supported. 83–100% vs. 0%; weight matching rules out permutation as the cause.
H3	The last task is forgotten least	Zero for the last task, but by construction. No independent evidence.
H4	Fisher trace depends on when a task was learned	Lowest trace marks the last task in 4/6 orders (total and backbone); full order 1/6.

Limitations

- **One training run** (seed 42). No estimate of run-to-run variation.
- **Only six orders.** One flipped prediction changes accuracy by 16.7 points.
- **Fisher predictions are concentrated on A** (4 of 6 orders).
- **Shared initialization.** Weight matching returns the identity, so its benefit cannot show here.
- **Small tasks.** 5 classes each, from CIFAR-100; the tasks may be too similar or too different.
- **No task boundaries or forgetting protection** in training; heads are always 5-class.
- **Hardware.** Seeds give deterministic results on the same hardware only.

Conclusions and next steps

Conclusions

- The last task is detectable in the **representations** (CKA, feature drift) and, less reliably, in the **curvature** (Fisher).
- Weight distance and update cosine do not reveal it; both point at the first task.
- Forgetting adds no independent evidence.

Next steps

- Repeat with several training seeds and aggregate.
- Use independently initialized references, where weight matching matters.
- Find out why weight-space measures favor the first task.
- Try other architectures, intermediate layers, other task sets, and more than three tasks.